

RECESSED MINDS

Demo: Protocolo Q-17

Karen Daniela Ruiz Madrigal

Juan Andres Quiceno Banoy

Andres Felipe Piñeros Lozano

Juan Sebastian Delgado Inocencio

MANUAL TÉCNICO

Videojuego de terror y narrativa 3D en primera persona

Proyecto:	Recessed Minds – Demo Protocolo Q-17
Versión:	1.0 – Demo Pública
Motor:	Unity 2022 LTS
Plataforma:	PC (Windows / Linux)
Fecha:	Junio 20 26

1. Introducción

1.1. Descripción General del Proyecto

Recessed Minds es un videojuego de terror psicológico y narrativa en primera persona desarrollado con Unity 2022 LTS. El proyecto está concebido como una experiencia interactiva que combina exploración de entornos 3D, resolución de puzzles ambientales y una narrativa progresiva que se revela a través de documentos, objetos y grabaciones dispersos en el escenario.

La demo pública lleva por nombre Protocolo Q-17 y constituye un nivel introductorio autónomo que presenta el universo del juego sin revelar los eventos centrales del título completo. Está diseñada para ejecutarse en plataformas PC bajo sistemas operativos Windows y Linux.

1.2. Propósito del Manual Técnico

Este documento describe la arquitectura del sistema, las tecnologías empleadas, los módulos que componen el juego, la estructura de datos, los flujos de la demo y los criterios de prueba. Está dirigido al equipo de desarrollo para garantizar la comprensión, mantenimiento y escalabilidad del proyecto.

1.3. Alcance de la Demo

La demo Protocolo Q-17 abarca el nivel completo de una universidad abandonada estructurado en tres zonas: Patio de entrada, Ala Izquierda y Ala Derecha. El jugador controla a un guardia nocturno con el objetivo aparente de recuperar el expediente Q-17 y verificar el estado de una muestra sellada, descubriendo paulatinamente que el accidente químico que cerró el edificio esconde versiones contradictorias y evidencia manipulada.

Objetivo narrativo de la demo

Encontrar el expediente Q-17 en Secretaria y recuperar la Muestra Q-17 del frigorifico antes de abandonar el edificio. El objetivo parece administrativo, pero introduce las tres capas narrativas del juego completo.

2. Arquitectura del Sistema

2.1. Visión General

El sistema se divide en cuatro capas principales: motor de juego (Unity), capa de presentación (frontend web en Astro + Preact), capa de lógica de negocio (backend en Node.js + Express) y capa de persistencia (MongoDB). La demo ejecuta únicamente la capa del motor; el resto del stack soporta la web del juego y el módulo de usuarios.

Capa	Tecnología	Responsabilidad
Motor de juego	Unity 2022 LTS + C#	Renderizado 3D, física, IA, audio, puzzles
Modelado 3D	Blender 3.x	Personajes, escenarios, objetos del nivel
Frontend web	Astro + Preact	Landing, login, calificaciones, notificaciones
Backend	Node.js + Express	API REST, autenticacion JWT, logica de sesion
Base de datos	MongoDB	Usuarios, progreso, puntajes guardados
Control de versiones	Git + GitHub	Repositorio del proyecto

2.2. Arquitectura Interna de Unity

El proyecto de Unity sigue el patron de arquitectura basado en componentes nativo del motor, complementado con el patron Singleton para los managers globales. Cada objeto del juego (GameObject) agrupa los componentes necesarios para su comportamiento, lo que facilita la reutilizacion y el mantenimiento del codigo.

Managers Globales (Singletons)

- GameManager: controla el estado general del juego (menu, en partida, pausado, fin).
- AudioManager: gestiona musica ambiental, efectos de sonido y transiciones de audio.
- InventoryManager: almacena y expone los objetos recolectados por el jugador.
- SaveManager: serializa y deserializa el estado de partida en disco.
- UIManager: administra las pantallas de HUD, menus y overlays de texto.

Sistemas Secundarios

- InteractionSystem: detecta objetos interactuables mediante raycast desde la camara del jugador.
- PuzzleSystem: evalua condiciones de activacion y desbloquea puertas o contenedores.
- EnemyAI: controla el comportamiento de patrulla y persecucion de las amenazas presentes en la demo.
- DialogueSystem: muestra textos de radio, notas y subtítulos en pantalla.

2.3. Arquitectura del Stack Web

La pagina web del juego fue migrada de HTML estatico a Astro con componentes interactivos en Preact. El servidor de backend expone una API REST protegida con JWT que gestiona autenticacion, sesiones y registro de puntajes. La base de datos MongoDB almacena colecciones de usuarios, sesiones y registros de progreso.

3. Tecnologías y Herramientas

3.1. Motor de Juego

Herramienta	Especificacion
Unity	2022.3 LTS – render pipeline URP (Universal Render Pipeline)
Lenguaje	C# (.NET 4.x, compatible con Unity)
IDE	Visual Studio Code con extension C# Dev Kit
Control de entrada	Unity Input System (paquete nuevo)

Fisica	Unity PhysX integrado
Audio	Unity Audio Mixer + FMOD (opcional)
Navegacion IA	Unity NavMesh Baking + NavMeshAgent

3.2. Modelado y Arte 3D

Herramienta	Uso
Blender 3.x	Modelado, rigging y animacion de personajes y escenarios
Formato exportacion	.fbx para importacion en Unity
Texturas	PNG / PSD; empaquetadas en Atlas para optimizar draw calls
Materiales	URP Lit Shader y Unlit Shader segun necesidad

3.3. Stack Web

Componente	Tecnologia
Framework frontend	Astro 4.x con islas de Preact
Estilos	CSS modular + variables globales
Backend	Node.js 20 LTS + Express 4.x
Autenticacion	JWT (jsonwebtoken) + bcrypt para hash de contraseñas
Base de datos	MongoDB 7.x + Mongoose ODM
Entorno	Variables de entorno con dotenv

3.4. Requisitos de Hardware

Componente	Minimo	Recomendado
CPU	Intel Core i5 4ta gen / AMD Ryzen 3	Intel Core i7 8va gen / AMD Ryzen 5 3600
RAM	8 GB	16 GB
GPU	NVIDIA GTX 960 / AMD RX 480	NVIDIA RTX 2060 / AMD RX 5700

Almacenamiento	4 GB disponibles (HDD)	4 GB disponibles (SSD)
SO	Windows 10 64-bit / Ubuntu 20.04	Windows 11 64-bit / Ubuntu 22.04
DirectX	DirectX 11	DirectX 12

4. Estructura del Proyecto

4.1. Repositorio Unity

El proyecto de Unity sigue la estructura de carpetas estandar recomendada para proyectos medianos. Cada carpeta agrupa activos por tipo para facilitar la navegacion y el control de versiones.

Ruta	Contenido
Assets/Scripts/	Todos los scripts C# del juego
Assets/Scripts/Managers/	GameManager, AudioManager, SaveManager, InventoryManager, UIManager
Assets/Scripts/Player/	PlayerController, PlayerInteraction, PlayerInventory
Assets/Scripts/Enemies/	EnemyAI, EnemyPatrol, EnemyDetection
Assets/Scripts/Puzzles/	PuzzleBase, DocumentPuzzle, CombinePuzzle, SwitchPuzzle
Assets/Scripts/Items/	Item, Collectible, KeyItem, Document
Assets/Models/	Modelos .fbx importados desde Blender
Assets/Textures/	Texturas PNG / Atlas por zona del nivel
Assets/Audio/	Musica ambiental, efectos de sonido, dialogos de radio
Assets/Scenes/	MainMenu.unity, Demo_Q17.unity, Loading.unity
Assets/Prefabs/	Prefabs reutilizables: puertas, items, enemigos, UI
Assets/UI/	Canvas, sprites de HUD, fuentes

4.2. Estructura del Backend

Archivo / Carpeta	Descripcion
server.js	Punto de entrada del servidor Express

routes/auth.js	Endpoints de registro e inicio de sesion
routes/users.js	Gestion de perfil y datos del usuario
routes/scores.js	Registro y consulta de puntajes
models/User.js	Schema Mongoose del usuario
models/Score.js	Schema Mongoose del puntaje
middleware/auth.js	Verificacion de token JWT
.env	Variables de entorno (MONGO_URI, JWT_SECRET, PORT)

5. Módulos del Sistema

5.1. Módulo de Jugador

El modulo del jugador encapsula el control de movimiento en primera persona, la gestion del inventario y el sistema de interaccion con objetos del entorno. El componente PlayerController gestiona el input de teclado y raton mediante el nuevo Input System de Unity, aplicando movimiento y rotacion de camara con aceleracion suave.

- PlayerController.cs: movimiento WASD, sprint con Shift, agacharse con Ctrl.
- PlayerInteraction.cs: raycast desde la camara con rango de 2 m; detecta IInteractable.
- PlayerInventory.cs: lista de Item; dispara eventos al agregar o combinar objetos.

5.2. Módulo de Inventario y Objetos

Los objetos del juego implementan la interfaz IItem con propiedades de identificador unico, nombre, descripcion e icono. Los objetos clave (KeyItem) bloquean puertas o puzzles hasta que esten en el inventario. La combinacion de objetos se resuelve mediante el CombineManager, que recibe dos IDs y retorna el objeto resultante si la combinacion existe en el diccionario de recetas.

Objeto	Tipo	Uso en la demo
Linterna	KeyItem	Iluminacion desde el inicio del nivel

Tarjeta admin. danada	KeyItem	Acceso a Secretaria tras validacion
Sello de validacion	Collectible	Se combina con la tarjeta
Tarjeta admin. funcional	KeyItem	Resultado de combinacion; abre Ala Derecha
Destornillador	KeyItem	Abre cajones y paneles en Salones
Manivela incompleta	KeyItem	Abre Frigorifico tras reparacion
Pieza metalica	Collectible	Encontrada en cabina del Auditorio
Cinta aislante	Collectible	Sala de Conserje; parte de reparacion
Manivela reparada	KeyItem	Resultado de combinacion triple
Fusible congelado	KeyItem	Abre Cocina desde el Frigorifico
Expediente Q-17	Document	Objetivo principal de Secretaria
Muestra Q-17	KeyItem	Objetivo final del Frigorifico
Cinta de seguridad Q-17	KeyItem	Revela la grabacion en el panel final
Llave del Panel	KeyItem	Abre el panel del Salon Principal

5.3. Módulo de Puzzles

El sistema de puzzles hereda de la clase abstracta PuzzleBase, que define los metodos Activate(), Solve() y OnSolved(). Cada subclase implementa su logica de validacion:

- DocumentPuzzle: ordena documentos o carpetas en una secuencia cronologica. Al colocar los tres elementos en orden correcto se dispara la apertura de la puerta destino.
- CombinePuzzle: recibe dos o tres items del inventario y retorna un nuevo objeto si la combinacion esta registrada en CombineManager.
- SwitchPuzzle: activa interruptores en un orden especifico. Si el orden es incorrecto se reinicia sin penalidad.
- SymbolPuzzle: asigna simbolos a posiciones (Ojo/Boca en Consejeria). Valida contra un array de respuestas correctas.

- LightPuzzle: activa focos del Auditorio en la secuencia correcta segun la hoja de evento encontrada.
- SafePuzzle: ingreso de codigo numerico (1722) en el armario del Almacen.

5.4. Módulo de Enemigos

La IA de los enemigos usa el componente NavMeshAgent de Unity para la navegacion por el nivel. El comportamiento se organiza en tres estados gestionados por una maquina de estados finita (FSM):

- Patrulla: el enemigo recorre puntos de waypoint predefinidos en el nivel. Velocidad reducida.
- Alerta: el enemigo detecta al jugador por campo de vision (trigger de colision conica) o por audio (radio de sonido configurable). Velocidad aumentada, animacion diferente.
- Persecucion: el enemigo persigue al jugador directamente. Si pierde el rastro, regresa a patrulla tras un tiempo de espera.

En la demo el enemigo no produce game over instantaneo en el primer contacto. Se aplica un sistema de dano por proximidad que reduce la salud del jugador. Si la salud llega a cero, se muestra la pantalla de muerte con opcion de reiniciar desde el ultimo punto de guardado.

5.5. Módulo de Guardado

El SaveManager serializa el estado de partida en un archivo JSON ubicado en Application.persistentDataPath. Los datos guardados incluyen: posicion del jugador, zona activa, lista de items en inventario, estado de cada puerta y puzzle, y documentos leidos. El punto de guardado de la demo se encuentra en la Recepcion (sala segura). El jugador puede guardar manualmente desde el baul de la sala segura.

6. Estructura del Nivel – Demo Protocolo Q-17

6.1. Mapa del Nivel

El nivel de la demo se organiza en cuatro zonas principales conectadas entre si. El flujo de juego sigue una progresion lineal con backtracking controlado.

Zona	Salas incluidas	Objeto/Progreso clave	Desbloquea
Entrada	Patio, Salon Principal, Recepcion	Linterna, Tarjeta danada, Llave Ala Izq.	Ala Izquierda
Ala Izquierda	Banos Prof., Salones, Sala Profesores, Sala Archivos, Auditorio	Destornillador, Llave Sala Prof., Sello de validacion, Pieza metalica	Tarjeta funcional, Ala Derecha, Manivela reparada
Ala Derecha	Secretaria, Consejeria, Comedor, Banos, Sala Conserje, Frigorifico, Cocina, Almacen	Expediente Q-17, Manivela incompleta, Llave Auditorio, Muestra Q-17, Cinta aislante, Fusible congelado, Cinta seguridad	Panel final del Salon Principal
Final	Salon Principal (panel)	Cinta Q-17 + Expediente + Muestra	Secuencia final y cierre de demo

6.2. Flujo de Progresion

La siguiente secuencia describe el orden de acciones del jugador a lo largo de la demo:

1. Inicio en el Patio. El jugador obtiene la Linterna y avanza al Salon Principal.
2. En Recepcion obtiene la Tarjeta danada y la Llave del Ala Izquierda.
3. En Banos de Profesores obtiene el Destornillador.
4. En Salones abre el escritorio con el destornillador. Obtiene Llave de Sala de Profesores.
5. En Sala de Profesores resuelve el puzzle de carpetas cronologicas. Se abre Sala de Archivos.
6. En Sala de Archivos obtiene el Sello de validacion y combina: Tarjeta danada + Sello = Tarjeta funcional.
7. Con la Tarjeta funcional accede al Ala Derecha (Secretaria).
8. En Secretaria obtiene el Expediente Q-17 incompleto y la Llave de Consejeria.

9. En Consejeria resuelve el puzzle de simbolos (Ojo/Boca). Obtiene Llave del Comedor.
10. En Comedor obtiene recursos. Accede a Banos del Ala Derecha.
11. En Banos obtiene la Manivela incompleta. Necesita pieza del Auditorio.
12. En Sala de Conserje obtiene la Llave del Auditorio y la Cinta aislante.
13. En Auditorio resuelve el puzzle de focos. Obtiene la Pieza metalica.
14. Combinacion: Manivela incompleta + Pieza metalica + Cinta aislante = Manivela reparada.
15. Regresa al Comedor y usa la Manivela reparada en el Frigorifico.
16. En Frigorifico obtiene la Muestra Q-17 y el Fusible congelado.
17. En Cocina usa el fusible en el panel electrico. Resuelve el puzzle de interruptores. Abre Almacen.
18. En Almacen obtiene la Cinta de seguridad Q-17 y la Llave del Panel usando el codigo 1722.
19. Regresa al Salon Principal. Inserta Cinta + Expediente + Muestra en el panel. Reproduce la grabacion.
20. Secuencia final. Corte a negro. Fin de la demo.

6.3. Sistema de Puertas y Bloqueos

Cada puerta del nivel tiene un componente Door con un estado (abierta, cerrada, bloqueada). El desbloqueo se realiza mediante tres metodos: presencia de KeyItem en el inventario, resolucion de un puzzle asociado, o activacion de un interruptor. Una vez desbloqueada, la puerta permanece abierta para el resto de la sesion.

7. Base de Datos

7.1. Modelo de Datos

La base de datos MongoDB contiene tres colecciones principales que soportan el modulo web y de usuarios del proyecto.

Coleccion: users

Campo	Tipo	Descripcion
-------	------	-------------

_id	ObjectId	Identificador unico generado por MongoDB
username	String	Nombre de usuario; unico en la coleccion
email	String	Correo electronico; unico en la coleccion
password	String	Hash bcrypt de la contrasena
createdAt	Date	Fecha de registro del usuario
role	String	Rol: 'user' o 'admin'

Coleccion: scores

Campo	Tipo	Descripcion
_id	ObjectId	Identificador unico
userId	ObjectId	Referencia al usuario (ref: 'User')
score	Number	Puntaje obtenido en la partida
level	String	Nombre del nivel jugado ('demo_q17')
time	Number	Tiempo en segundos de la partida
createdAt	Date	Fecha del registro

Coleccion: saves

Campo	Tipo	Descripcion
_id	ObjectId	Identificador unico
userId	ObjectId	Referencia al usuario
zone	String	Zona activa al guardar
inventory	Array	Lista de IDs de items en inventario
puzzleStates	Object	Mapa puzzleId → estado (resuelto/pendiente)
doorStates	Object	Mapa doorId → estado (abierta/cerrada)
playerPos	Object	Coordenadas {x, y, z} del jugador
savedAt	Date	Marca de tiempo del guardado

8. API REST – Backend

8.1. Endpoints de Autenticacion

Metodo	Endpoint	Body	Respuesta
POST	/api/auth/register	{ username, email, password }	201: { user, token }
POST	/api/auth/login	{ email, password }	200: { user, token }
GET	/api/auth/me	— (Header: Bearer token)	200: { user }

8.2. Endpoints de Puntajes

Metodo	Endpoint	Body	Respuesta
POST	/api/scores	{ score, level, time }	201: { scoreDoc }
GET	/api/scores/top	— (Query: ?level=demo_q17)	200: [{ username, score, time }]
GET	/api/scores/me	— (Header: Bearer token)	200: [{ score, level, time }]

8.3. Seguridad

- Todos los endpoints protegidos requieren el header Authorization: Bearer <token>.
- El middleware auth.js verifica la firma del JWT y rechaza solicitudes con token expirado o invalido.
- Las contraseñas se almacenan exclusivamente como hash bcrypt con factor de costo 12.
- El servidor implementa CORS restringido al dominio del frontend y limita peticiones con express-rate-limit.

9. Interfaz Web – Frontend Astro

9.1. Estructura de Componentes

La pagina web del juego fue migrada de un HTML estatico a un proyecto Astro con islas de interactividad en Preact. La arquitectura de componentes sigue el patron Layout → Secciones → Componentes interactivos.

Componente	Funcion
------------	---------

Layout.astro	Plantilla base: meta, fuentes, header y footer globales
Nav.astro	Barra de navegacion principal con links a secciones
Hero.astro	Seccion de presentacion del juego con trailer embed
LoginForm.jsx (Preact)	Formulario de inicio de sesion con validacion y llamada a API
RatingWidget.jsx (Preact)	Sistema de calificacion de 1 a 5 estrellas
NotificationsPanel.jsx (Preact)	Panel de notificaciones del usuario autenticado
Leaderboard.jsx (Preact)	Tabla de mejores puntajes obtenida desde la API
Footer.astro	Pie de pagina con creditos del equipo

9.2. Optimizacion UI/UX

- Las imagenes del sitio usan el componente <Image> de Astro para conversion automatica a formato WebP.
- Los componentes Preact se cargan con directiva client:visible para diferir la hidratacion hasta que sean visibles en pantalla.
- El sitio obtiene una puntuacion de accesibilidad superior a 90 en Lighthouse mediante atributos ARIA y contraste de color validado.
- Los estilos globales y las variables de color se definen en un archivo CSS separado importado en Layout.astro.

10. Plan de Pruebas

10.1. Pruebas Funcionales

Las pruebas funcionales validan que cada mecanica del juego opere segun lo especificado en los requisitos. Se ejecutan manualmente por el equipo de testers siguiendo una lista de casos de prueba predefinidos.

ID	Caso de prueba	Resultado esperado	Estado
F01	Recoger linterna al inicio	Item aparece en inventario	Pendiente
F02	Combinar tarjeta danada + sello	Genera tarjeta funcional	Pendiente

F03	Resolver puzzle de carpetas	Abre puerta a Sala de Archivos	Pendiente
F04	Puzzle simbolos Consejeria (Ojo-A, Boca-B)	Entrega llave del Comedor	Pendiente
F05	Puzzle focos Auditorio en orden correcto	Abre cabina tecnica	Pendiente
F06	Combinar manivela + pieza + cinta	Genera manivela reparada	Pendiente
F07	Usar manivela reparada en frigorifico	Puerta del frigorifico se abre	Pendiente
F08	Puzzle interruptores Cocina	Desbloquea acceso al Almacen	Pendiente
F09	Codigo 1722 en armario Almacen	Entrega llave del Panel	Pendiente
F10	Insertar Cinta+Expediente+Muestra en panel	Reproduce grabacion final	Pendiente
F11	Guardar partida en sala segura	Estado se persiste en disco	Pendiente
F12	Cargar partida guardada	Restaura inventario, puertas y posicion	Pendiente

10.2. Pruebas de Rendimiento

Se verificara que el juego mantenga una tasa de fotogramas estable en los equipos de especificacion minima. Las metricas objetivo son:

- Minimo 30 FPS sostenidos en hardware minimo (GTX 960 / 8 GB RAM) en resolucion 1080p.
- Tiempo de carga del nivel inferior a 30 segundos en almacenamiento HDD.
- Uso de VRAM inferior a 2 GB en ajustes de calidad media.
- Sin memory leaks detectables tras 30 minutos de sesion continua (verificado con Unity Profiler).

10.3. Pruebas de Integracion

Se verificara la comunicacion entre los modulos internos de Unity y entre el cliente web y el backend:

- InventoryManager notifica correctamente al UIManager al agregar o combinar items.
- SaveManager persiste y restaura el estado sin corrupcion tras cierre forzado del proceso.
- La API de puntajes registra y devuelve datos correctos tras una sesion de juego completada.
- El token JWT es validado correctamente en todos los endpoints protegidos del backend.

10.4. Pruebas de Usabilidad

Se realizaran sesiones de prueba con usuarios externos al equipo de desarrollo. Se mediran indicadores de comprension de los puzzles, claridad de los objetivos en pantalla y comodidad de los controles. Los resultados se documentaran en el informe de pruebas de usabilidad y guiaran los ajustes previos a la entrega final.

11. Despliegue y Entrega

11.1. Build del Videojuego

El build de la demo se genera desde Unity mediante File > Build Settings. Se selecciona la plataforma PC, Mac & Linux Standalone con arquitectura x86_64. El ejecutable resultante se distribuye como archivo comprimido .zip que incluye el ejecutable, la carpeta de datos del juego y las librerias nativas.

- Build de Windows: genera RecssedMinds_Demo.exe + carpeta RecssedMinds_Demo_Data/.
- Build de Linux: genera RecssedMinds_Demo.x86_64 + carpeta RecssedMinds_Demo_Data/.
- Compresion: archivo .zip nombrado RecssedMinds_Demo_v1.0_[plataforma].zip.

11.2. Pruebas de Instalacion

Antes de la entrega final se verifica la instalacion en una maquina limpia (sin Unity instalado) con las especificaciones minimas del proyecto. Se comprueba que el ejecutable

inicia correctamente, que el audio funciona y que el guardado escribe en la ruta de datos del usuario sin requerir permisos de administrador.

11.3. Configuración del Entorno de Producción Web

- El backend se despliega en un servidor con Node.js 20 LTS.
- Las variables de entorno (MONGO_URI, JWT_SECRET, PORT) se configuran en el servidor sin commitearlas al repositorio.
- El frontend Astro se genera con `npm run build` y se sirve como archivos estáticos desde un servidor Nginx o Vercel.
- La base de datos MongoDB se aloja en MongoDB Atlas (tier gratuito para el entorno de desarrollo).

11.4. Entregables Finales

Entregable	Descripción
Build ejecutable (Windows/Linux)	Archivo .zip con la demo lista para ejecutar
Repositorio GitHub	Código fuente completo del proyecto Unity y web
Manual técnico	Este documento
Manual de usuario	Guía de instalación y controles para el jugador
Informe de pruebas	Resultados de pruebas funcionales, rendimiento y usabilidad
Documentación de la API	Especificación de endpoints del backend

12. Glosario

Término	Definición
API REST	Interfaz de programación que permite la comunicación entre cliente y servidor mediante peticiones HTTP estándar.
Astro	Framework web para generación de sitios estáticos con soporte de islas de interactividad en frameworks de JavaScript.

bcrypt	Algoritmo de hash diseñado para el almacenamiento seguro de contraseñas, con factor de costo configurable.
FSM	Maquina de estados finita; modelo de comportamiento que define estados y transiciones entre ellos.
GameObject	Entidad base de Unity que actua como contenedor de componentes.
JWT	JSON Web Token; estandar de autenticacion que codifica informacion en un token firmado digitalmente.
NavMesh	Malla de navegacion generada en Unity que define las areas transitables por agentes de IA.
ODM	Object Document Mapper; biblioteca que mapea documentos de base de datos a objetos de codigo (Mongoose para MongoDB).
Prefab	Activo reutilizable de Unity que encapsula un GameObject y sus componentes para instanciarse multiples veces.
Preact	Alternativa ligera a React con la misma API; usada para componentes interactivos en Astro.
Protocolo Q-17	Nombre interno del accidente quimico que cierra la universidad; tambien nombre de la demo.
Raycast	Tecnica de deteccion de colisiones que proyecta un rayo en el espacio 3D para identificar objetos.
Singleton	Patron de diseno que garantiza que una clase tenga una sola instancia global accesible.
URP	Universal Render Pipeline; pipeline de renderizado de Unity optimizado para una amplia gama de plataformas.